FEL User Guide

FEL is a debiasing framework for feature-based binary classification using neural networks. It mitigates bias during model training by optimizing fairness axioms alongside standard performance objectives. This guide uses the COMPAS dataset, predicting whether a person will reoffend within 2 years, as a running example.

Key concepts

Before using FEL, you need to define two things about your prediction task.

The target variable is the binary outcome the model predicts. One class must be designated as favourable (an outcome the individual wants to receive, e.g. no reoffending) and the other as unfavourable (an outcome to avoid, e.g. reoffending within 2 years). Which class is which cannot be derived from the data alone — it requires domain knowledge about the task.

The sensitive feature is a binary input attribute that should not drive discriminatory predictions (e.g. sex, age group). Its values are labelled as privileged (the group historically receiving better outcomes) or unprivileged (the group at risk of disadvantage).

Both the target variable and the sensitive feature must be binary. Continuous or multi-category attributes must be discretized before use.

Training

Step 0) Deployment

Download the code from github (https://github.com/unimib-datAI/FEL) and follow the deployment instructions.

Step 1) Prepare the dataset

The input must be a CSV file with a header row. All values must be numerical (integer or float). Apply the following transformations to your raw data before passing it to FEL:

- Categorical features → one-hot encoding
- Boolean features → integers (False = 0, True = 1)
- Sensitive features that are not already binary (e.g. age, sex with multiple categories) → binarize and rename accordingly (e.g. sex → sex_male, with 1 = male, 0 = female)
- Missing values → impute before passing the file to FEL (column mean is a simple fallback)

Step 2) Configure FEL

Create a config.yaml file in the project root with the following structure:

```yaml
data:
  target_variable: "two_year_recid" # Your target column name
  sensitive_feature: "sex"          # Protected attribute column
  labels:
    favourable: 0    # no receid
    unfavourable: 1  # did receid
  protected_values:
    privileged: 1    # female
    unprivileged: 0  # male

model:
  hidden_layer_sizes: [50, 50]
  implies: "Godel"  # Options: KleeneDienes, Godel, Reichenbach, Goguen, Luk
  p_mean: 1
  aggregator_deviation: 2

fairness:
  enabled: true
  weight: 0.5

training:
  epochs: 200
```

The fields to set are:

data section:

- target_variable: column name of the target in the CSV
- sensitive_feature: column name of the sensitive attribute
- labels → favourable / unfavourable: the values of the target variable corresponding to each outcome
- protected_values → privileged / unprivileged: the values of the sensitive feature identifying each group

model section:

- hidden_layer_sizes: number of neurons per hidden layer
- implies: fuzzy implication operator (KleeneDienes, Godel, Reichenbach, Goguen, or Luk)
- p_mean: quantifier application operator
- aggregator_deviation: deviation control in the aggregator

fairness section:

- enabled: set to true to activate the fairness axiom; set to false to run FEL as a standard classifier without fairness constraints (useful for establishing a performance baseline or for debugging)
- weight: relative weight of the fairness axiom with respect to the performance axiom

training section:

- epochs: number of training epochs

Step 3) Run training:

The training of the model will take as an input the dataset path and the configuration.
Once the process is concluded, the model folder will be created containing:

At the end of training, the models/ folder will contain:

- kb.npz: the trained model
- test_metrics.png: plots of performance and fairness metrics over the test split across training epochs
- prediction plots for the test split at the end of training

Inference:

Step 1) Prepare the test dataset

Apply the same preprocessing as for training: same column names, same encoding, same binarization choices.

Step 2) Run inference

By invoking the src/inference.py script, you can both evaluate the already trained model (models/kb.npz) in case you have a test set with ground truth, or you can use the trained model as a predictor.
You have to specify the following arguments:

- --config: path to the configuration file
- --dataset: path to the test dataset
- --model: path to the trained model (models/kb.npz)

Predictions will be saved in the models folder along with plots (if gt is provided)